11.2.1 reverse_merge and reverse_merge!

In case of collision the key in the hash of the argument wins in `merge`. You can support option hashes with default values in a compact way with this idiom:

```
options = {length: 30, omission: "..."}.merge(options)
```

Active Support defines `reverse_merge` in case you prefer this alternative notation:

```
options = options.reverse_merge(length: 30, omission: "...")
```

And a bang version `reverse_merge!` that performs the merge in place:

```
options.reverse_merge!(length: 30, omission: "...")
```

Take into account that `reverse_merge!` may change the hash in the caller, which may or may not be a good idea.

Defined in `active_support/core_ext/hash/reverse_merge.rb`.

11.2.2 reverse_update

The method `reverse_update` is an alias for `reverse_merge!`, explained above.

Note that `reverse_update` has no bang.

Defined in `active_support/core_ext/hash/reverse_merge.rb`.

11.2.3 deep_merge and deep_merge!

As you can see in the previous example if a key is found in both hashes the value in the one in the argument wins.

Active Support defines `Hash#deep_merge`. In a deep merge, if a key is found in both hashes and their values are hashes in turn, then their *merge* becomes the value in the resulting hash:

```
{a: {b: 1}}.deep_merge(a: {c: 2})
# => {:a=>{:b=>1, :c=>2}}
```

The method `deep_merge!` performs a deep merge in place.

Defined in `active_support/core_ext/hash/deep_merge.rb`.

11.3 Deep duplicating

The method `Hash.deep_dup` duplicates itself and all keys and values inside recursively with Active Support method `Object#deep_dup`. It works like `Enumerator#each_with_object` with sending `deep_dup` method to each pair inside.

```
hash = { a: 1, b: { c: 2, d: [3, 4] } }

dup = hash.deep_dup
dup[:b][:e] = 5
dup[:b][:d] << 5

hash[:b][:e] == nil   # => true
hash[:b][:d] == [3, 4]   # => true
```

Defined in `active_support/core_ext/object/deep_dup.rb`.

11.4 Working with Keys

11.4.1 except and except!

The method `except` returns a hash with the keys in the argument list removed, if present:

```
{a: 1, b: 2}.except(:a) # => {:b=>2}
```

If the receiver responds to `convert_key`, the method is called on each of the arguments. This allows `except` to play nice with hashes with indifferent access for instance:

```
{a: 1}.with_indifferent_access.except(:a)  # => {}
{a: 1}.with_indifferent_access.except("a") # => {}
```